四川大学计算机学院、软件学院

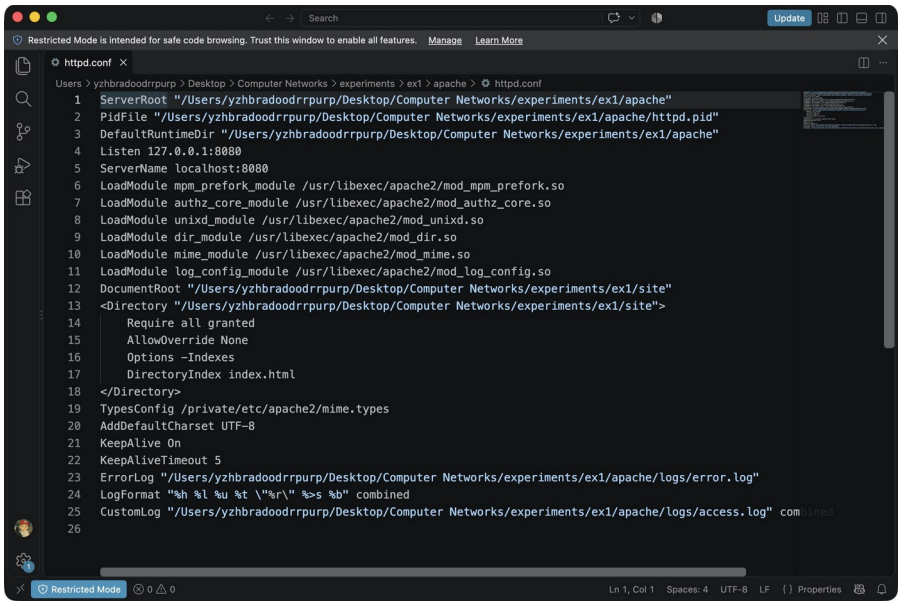
实 验 报 告

学号：2023141220023 姓名：易治行

专业：计算机科学与技术 班级：102 班 第 7 周

课程名称	计算机网络	**实验课时**	
实验项目	实验 1 Web 服务器搭建与 HTTP 协议分析	**实验时间**	2026 年 10 月 19 日
实验目的	本实验为综合型实验，结合 Web 服务器配置和 HTTP 协议验证。 1. 搭建 Apache 静态网站，理解监听地址、端口、网站根目录与默认首页的映射关系。 2. 使用 Wireshark 捕获真实 HTTP 请求和响应，分析方法、状态、首部、实体长度及连接复用。 3. 观察浏览器缓存与 Conditional GET，区分本地缓存命中、200 响应和 304 缓存验证。		
实验环境	硬件与系统：Apple Silicon Mac，macOS 26.5.1。 服务器：系统自带 Apache HTTP Server 2.4.66（/usr/sbin/httpd）。 浏览器：Chromium 153.0.0.0 独立临时会话；Safari 26.5 辅助验证首页。 工具：Wireshark dumpcap 4.6.9 捕获，TShark 4.6.9 解析，Wireshark 4.4.14 显示截图。 本机网站：http://127.0.0.1:8080/；目标服务器：gaia.cs.umass.edu，TCP 80 端口。		

实验内容（算法、程序、步骤和方法）	**一 Apache 网站搭建** 实验流程：独立配置 Apache → 语法检查与启动 → 浏览器访问自建网页 → 指定站点 HTTP 抓包 → 请求响应分析 → 缓存访问与刷新对比。 采用实验目录内的独立配置启动系统自带 Apache。自建 index.html 显示姓名、学号、班级及实验名称，标题为“计算机网络课程设计——实验 1”。  图 1 浏览器通过 HTTP 访问 Apache 默认首页 启动前执行 httpd -t -f 配置文件，得到 Syntax OK；启动后浏览器与 curl 均收到 HTTP/1.1 200 OK，响应正文与自建首页文件一致。 127.0.0.1 指向本机，8080 为 TCP 监听端口；路径 / 先映射到 DocumentRoot，再由 DirectoryIndex 选择 index.html。使用该端口时 URL 必须显式写出 :8080。

（接上） 实验内容	二 服务器配置与正式捕获  图 2 实际加载的 Apache 配置文件 Listen 127.0.0.1:8080 限定本机监听；DocumentRoot 指向 experiments/ex1/site，DirectoryIndex index.html 指定首页。Require all granted 允许读取网站，KeepAlive On 启用持久连接。 配置保存在 apache/httpd.conf，首页保存在 site/index.html。网站根目录为本实验目录下的 site；浏览器和服务器在同一台计算机运行。 正式捕获采用 Wireshark 自带 dumpcap，在 en0 上使用过滤器 host gaia.cs.umass.edu and tcp port 80。新的临时浏览器会话以 --no-proxy-server 直连，仅影响本次会话。 正式文件 evidence/wireshark-browser.pcapng 共 39 个分组。图 3 至图 8 的帧号均对应此文件；此前 tcpdump 预实验文件单独保存。

（接上） 实验内容	三 缓存操作与排障过程 基本资源：http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html 缓存资源：http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html 1. 以全新的临时浏览器会话访问 file1，再首次访问 file2，确保首次访问前没有该目标的既有缓存。 2. 首次加载 file2 后立即再次导航至同一 URL，观察是否出现新的目标请求，并记录导航计时。 3. 随后进行普通刷新，观察 If-None-Match、If-Modified-Since 与 304 响应；未使用硬刷新或手动构造首部。 **操作示意：首次 GET → 200 与 371 字节正文 → 保存缓存 → 再次导航直接使用缓存 → 普通刷新发送条件 GET → 304 后复用原正文。** 权限问题：抓包设备 /dev/bpf0 的所有者为 root，当前账户直接执行 tcpdump 得到 Permission denied。先以管理员权限运行定向 tcpdump 验证；Wireshark 安装完成后，再由本人在终端启动其自带 dumpcap 正式捕获。成功生成 pcapng，并由 Wireshark/TShark 识别请求、响应及条件字段，证明捕获链路有效。未修改全局 BPF 权限。 显示问题：Wireshark 4.6.9 的界面在自动读取辅助功能结构时出现 EXC_BAD_ACCESS，TShark 可正常解析同一文件。改用官方 Wireshark 4.4.14 查看原始 pcapng，界面稳定；截图的帧号和载荷与 4.6.9 解析结果一致。 验证：Wireshark 截图、TShark 解析和原始流记录对应同一正式抓包。通过 frame.number == 帧号可复核每条答案，缓存导航计时保存在 final-second-visit-cache.txt。

数据记录和计算

一 基本 HTTP 请求与响应

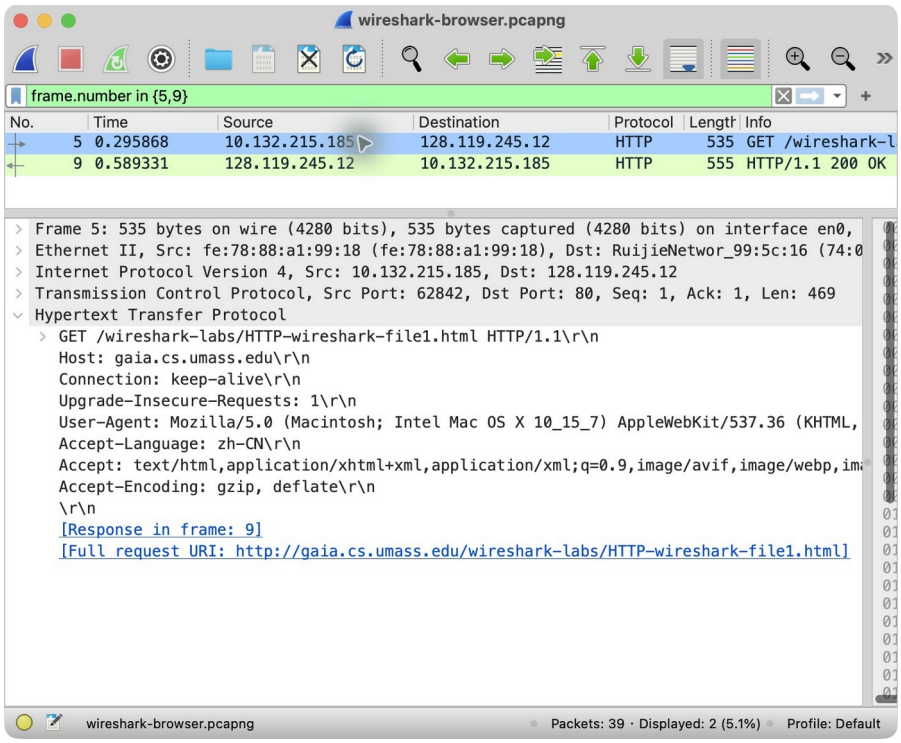


图 3 基本 GET 请求 第 5 帧

1．请求方法、资源与 HTTP 版本是什么?

图 3 第 5 帧请求行为 GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1，方法是 GET，资源为上述 HTML 路径，版本是 HTTP/1.1。Host 为 gaia.cs.umass.edu。

2．响应版本、状态码与状态短语是什么?

图 4 第 9 帧状态行为 HTTP/1.1 200 OK，版本为 HTTP/1.1，状态码为 200，状态短语为 OK。

3．客户端、服务器的 IP 与 TCP 端口分别是什么?

图 3 的 IPv4 与 TCP 摘要显示：客户端 10.132.215.185:62842，服务器 128.119.245.12:80；图 4 响应方向相反，证明两者属于同一连接。客户端地址是抓包接口上看到的局域网地址，不将其当成公网出口地址。

（接上）数据记录和计算

一 基本 HTTP 请求与响应 续

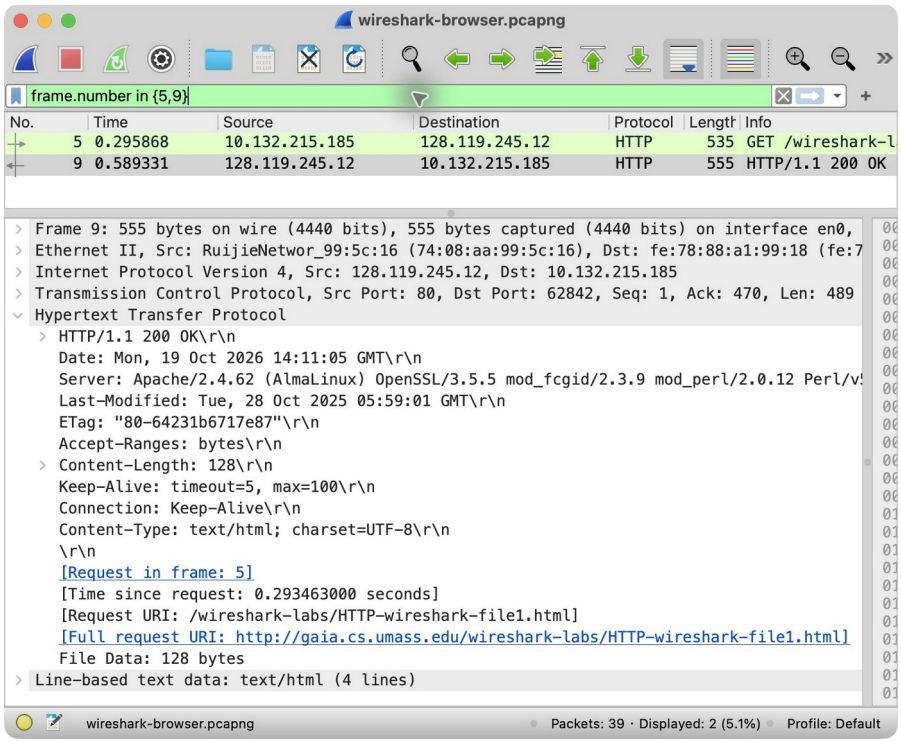


图 4 基本 200 响应 第 9 帧

4．Accept-Language 包含什么语言偏好?

图 3 显示 Accept-Language: zh-CN，表示客户端偏好中国大陆使用的中文。该值来自本次独立浏览器会话的语言设置，不代表服务器页面实际使用的语言。

5．Accept-Encoding 与实际编码有什么区别?

图 3 显示 Accept-Encoding: gzip, deflate，表示浏览器能够接受的内容编码。图 4 未出现 Content-Encoding，且正文可直接读到 HTML，说明本次响应没有通过该首部声明 gzip 或 deflate 等内容编码；不能从请求字段推断响应实际被压缩。UTF-8 是字符编码，也不等同于 gzip 内容编码。

6．Accept 与 Content-Type 的含义及区别是什么?

图 3 的 Accept 包含 text/html、application/xhtml+xml、application/xml;q=0.9，以及 image/avif、image/webp、image/apng、*/*;q=0.8 和 application/signed-exchange;v=b3;q=0.7。它表达客户端可接受的媒体类型及相对偏好；图 4 的 Content-Type: text/html; charset=UTF-8 则说明服务器实际返回的是 UTF-8 编码的 HTML。

（接上）数据记录和计算	**一 基本 HTTP 请求与响应 续二** **7．资源最后修改时间通过什么字段获得?** 图 4 显示 Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT，因此服务器声明的最后修改时间为 2025 年 10 月 28 日 05:59:01 UTC，即北京时间 13:59:01。Date 表示本次响应生成的时间，不能替代 Last-Modified。 **8．响应内容长度是多少，是报文总长还是实体长度?** 图 4 的 Content-Length: 128 表示响应内容为 128 字节，不包含 HTTP 状态行与首部，也不是 Ethernet/IP/TCP 帧的总长度。第 9 帧的 TCP 负载长度为 489 字节，包含 HTTP 首部与正文，因而大于 128 字节。 **9．如何确认响应属于目标网页?** 图 3 请求 URI 是 /wireshark-labs/HTTP-wireshark-file1.html，Host 是 gaia.cs.umass.edu；图 4 的 Request in frame 指向第 5 帧，正文中也含有目标 file1.html 的下载成功文字。与之区分，第 21 帧请求 /favicon.ico，第 23 帧响应 301，两者不是目标 HTML 的请求响应。 **10．连接是否允许保持，仅一组报文能否证明复用?** 请求和响应均为 HTTP/1.1，且包含 Connection: keep-alive，响应还有 Keep-Alive: timeout=5, max=100，说明允许连接保持。一组请求响应只能证明这次交换，不能单独证明复用；本次进一步检查同一 tcp.stream=0，发现第 5 帧的页面 GET 与第 21 帧的 favicon GET 使用同一客户端端口 62842，中间没有建立新的 TCP 连接，因此整段抓包确实提供了连接复用证据。 **缓存访问记录** 首次访问：22:11:05；第 11 帧 GET，第 13 帧 200；正文 371 字节 再次打开同一 URL：22:11:06；以同一地址再次导航，未出现新的目标 HTTP 请求；浏览器导航计时 transferSize=0，encodedBodySize=371 补充普通刷新：22:11:06；第 25 帧条件 GET，第 28 帧 304；无正文

（接上）
数据记录
和计算

二 Conditional GET 首次请求

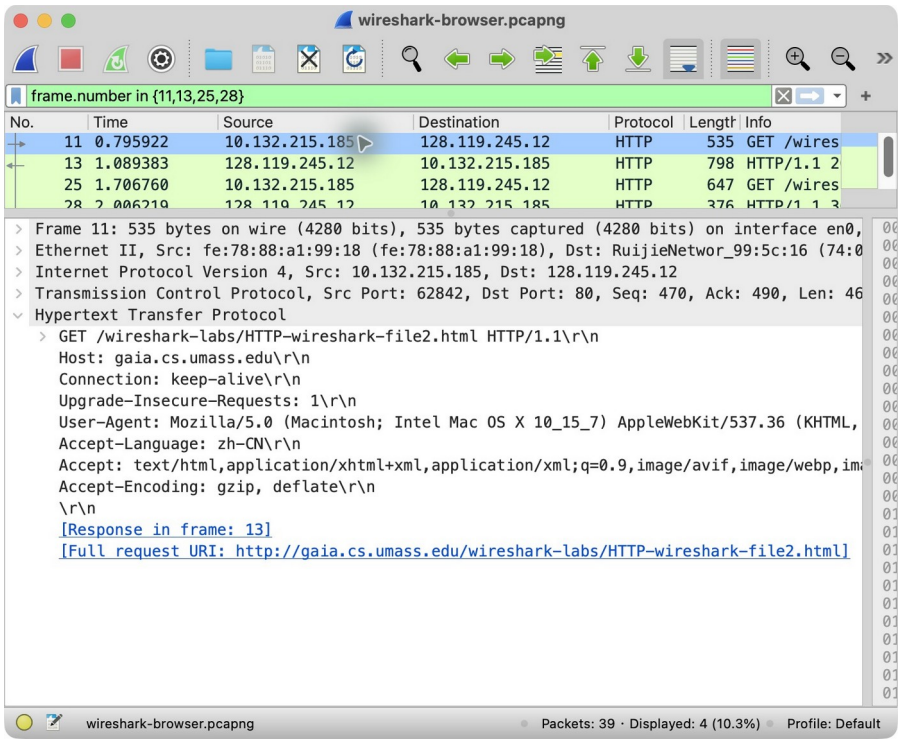


图 5 首次 GET 第 11 帧 没有缓存验证字段

1．第一次 GET 是否包含缓存验证字段?

图 5 第 11 帧中没有 If-Modified-Since 或 If-None-Match。该会话此前没有目标资源缓存，因而没有已保存的验证器可以附带，首次采用无条件 GET；图 6 第 13 帧返回 200 和 371 字节正文。

2．第一次响应包含哪些缓存验证字段?

图 6 包含 Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT，以及 ETag: "173-64231b67176b7"。前者提供资源最后修改时间，可用于后续 If-Modified-Since；后者提供该表示的实体标签，可用于 If-None-Match。两者都来自实际响应。

（接上）数据记录和计算

二 Conditional GET 首次响应与缓存导航

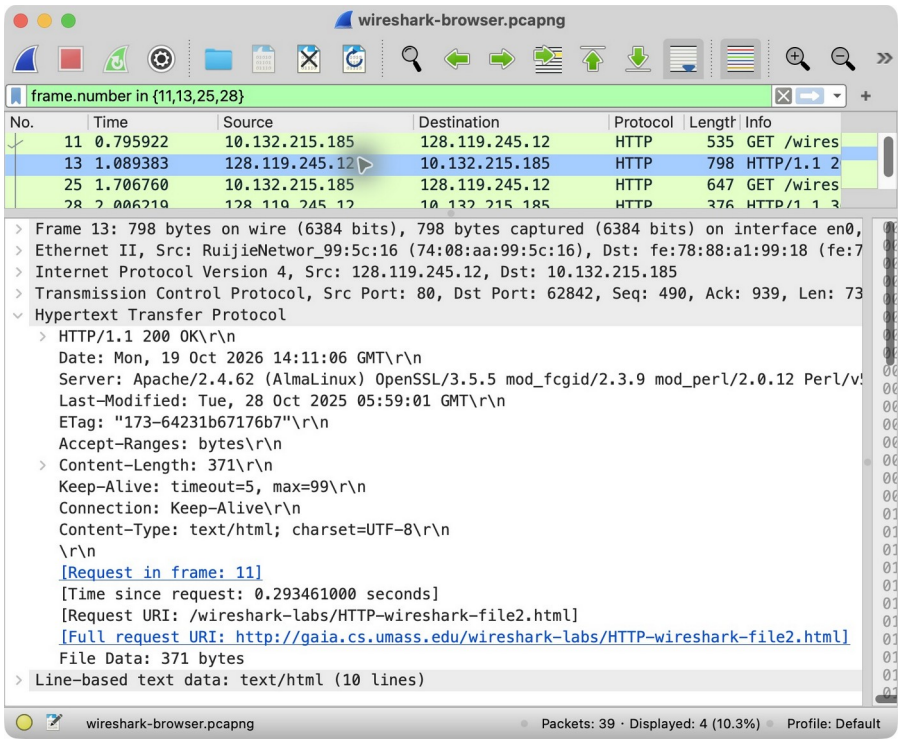


图 6 首次 200 响应 第 13 帧 371 字节正文与两个验证器

3．第二次访问是否发出了 GET，验证器如何关联?

第二次通过同一 URL 再次导航时，没有捕获到新的目标 GET；同时浏览器导航计时显示 transferSize=0、encodedBodySize=371，支持直接使用本地缓存。为排除抓包接口失效、HTTPS 升级、代理路径变化与请求被漏选，核查了同一捕获内的首次明文交换及随后刷新的明文交换、目标 80 端口过滤器、直连启动参数和实际导航 URL。补充普通刷新时，第 25 帧才出现 GET，If-None-Match 与首次响应的 ETag 完全相同，If-Modified-Since 与首次响应的 Last-Modified 完全相同，见图 7。

再次普通导航的计时为 transferSize=0、encodedBodySize=371、decodedBodySize=371；同一捕获中没有新的目标 GET。这与本地缓存复用相符。

（接上）
数据记录
和计算

二 Conditional GET 普通刷新请求

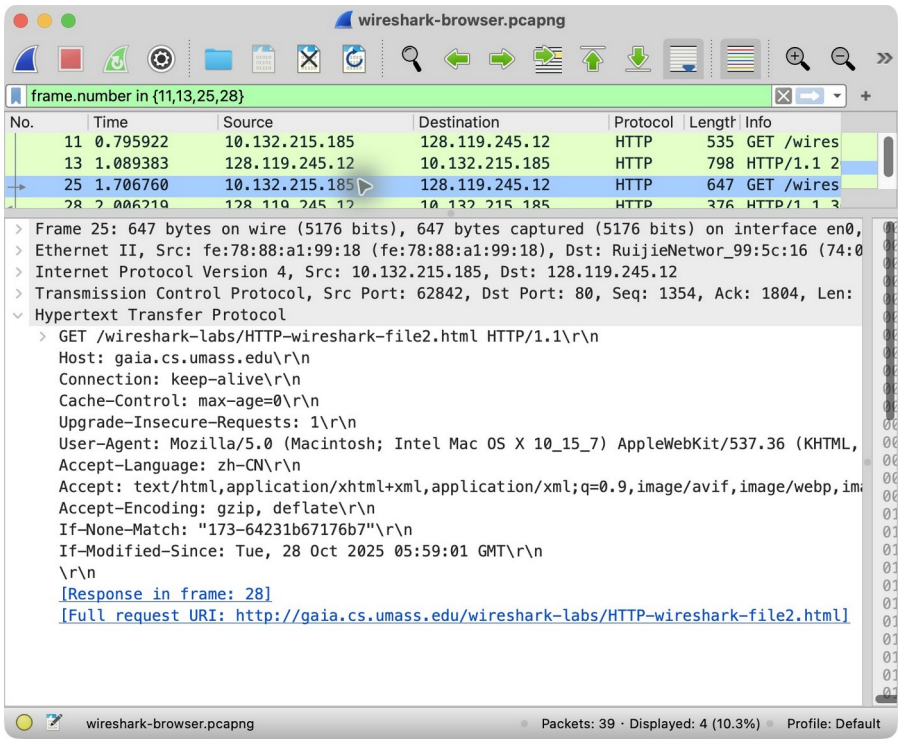


图 7 补充普通刷新 第 25 帧 条件 GET

4．为何未出现请求或返回 304，对传输有什么影响?

第二次普通访问没有发生本次网络缓存验证，也未重新传输正文；首次响应未给出显式 Cache-Control/Expires，但具有较早的 Last-Modified，允许浏览器采用启发式新鲜度，本地缓存命中与此相符，具体内部新鲜度算法未由抓包证明。补充刷新请求包含 Cache-Control: max-age=0 及两个条件字段，服务器在第 28 帧返回 304 Not Modified，确认现有表示可继续使用，未附带正文。由此避免再次传输 371 字节正文；304 仍有首部和 TCP 等开销，不能把节省的正文长度等同于全部网络字节节省量。

If-None-Match: "173-64231b67176b7" 与首次响应的 ETag 相同；If-Modified-Since: Tue, 28 Oct 2025 05:59:01 GMT 与 Last-Modified 相同。RFC 9110 规定两者同时出现时优先评估 If-None-Match。

（接上）
数据记录
和计算

二 Conditional GET 304 响应与长度比较

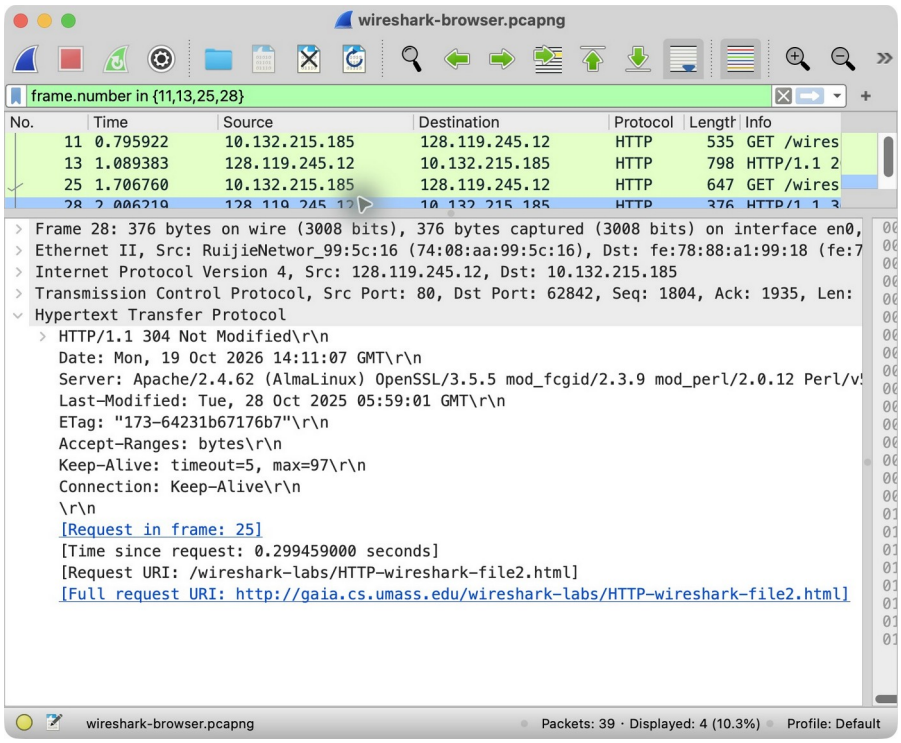


图 8 第 28 帧 HTTP/1.1 304 Not Modified

第 28 帧的 Request in frame 对应第 25 帧。HTTP 首部结束后没有 HTML 正文，TCP 负载为 310 字节；本次没有 Content-Length 首部，不能补写为 Content-Length: 0。

长度比较：file1 的 Content-Length 为 128 字节，对应 TCP 负载 489 字节；file2 首次 200 正文 371 字节，对应 TCP 负载 732 字节；后续 304 正文为空，对应 TCP 负载 310 字节。

本次 304 避免重新发送 371 字节正文。它仍传输了响应首部，链路上还存在 TCP/IP 等开销，因此不能把 371 字节直接当成总网络流量节省量。

结 论 （结 果）	**实验结果** 1. Apache 静态网站搭建成功。配置语法检查为 Syntax OK，浏览器根路径访问得到 200 响应，页面显示正确的姓名、学号、班级与实验名称，证实根目录和默认首页配置生效。 2. 正式捕获中的第 5/9 帧分别为目标 file1 的 GET/200 响应。请求与响应的版本、端点、媒体类型、正文长度及最后修改时间均可从实际首部核验。Accept-Encoding 仅表达客户端能力，不能证明返回内容被压缩。 3. 首次访问 file2 时第 11/13 帧为无条件 GET/200，保存 371 字节正文及两个验证器。立即再次导航未出现新的目标请求；普通刷新才在第 25/28 帧形成条件 GET/304，没有重新传输正文。 4. 同一 tcp.stream=0 中包含目标页面和 favicon 等多个请求，客户端端口为 62842，证实本次存在 TCP 连接复用。缓存命中与连接复用是不同机制，应分别依据报文和导航记录判断。
小 结	**1．依据哪些配置或抓包证据确认或修正原有判断?** Apache 配置中的 DocumentRoot、DirectoryIndex 与浏览器根路径访问结果共同确认了默认首页映射。再次普通访问没有网络请求，而补充刷新才出现条件 GET 和 304，这修正了“重复打开页面必然返回 304”的判断。 **2．若供他人使用，重点核验什么，资料查阅在哪一步有帮助?** 应核验监听地址是否包含可达网卡、访问控制和防火墙是否允许预期用户访问，并从另一台计算机验证实际 URL 与页面身份。本次查阅 Apache 官方的 DocumentRoot 与 DirectoryIndex 文档帮助完成路径映射；HTTP 缓存规范帮助区分本地新鲜缓存和 304 验证，查明重复访问未发请求的解释边界。
指导老师 评 议	成绩评定：　　指导教师签名：